

AI-Assisted Master Prompt Generation. Challenges and Proposed Solutions

The problem that I will address is:

When developers use AI coding agents over many prompts, after their developing process is done, they want to consolidate the entire prompt history into one master prompt at the end of the project to hand it over to a different person or AI model. As a result, they ask an AI agent to perform this consolidation which does not reliably return the original intent. Instead, the agent tends to presents its own implementation choices such as design pattern, specific filenames, or extra features as if they were user requirements.

Although modern AI coding agents can consolidate relatively short prompt histories when provided with carefully engineered instructions, it remains unclear whether these methods preserve requirement traceability and scale to realistic software development sessions. This thesis proposes a traceability-constrained prompt consolidation pipeline that separates user intent from implementation artifacts and evaluates its effectiveness across multiple AI coding agents.

Related Works

These papers are what I have read in the last few days to firstly figure out that these problems existed from the previous research. Secondly,

Most related in idea

- **Enhancing User Interaction in ChatGPT: Characterizing and Consolidating Multiple Prompts for Issue Resolution** — Mondal, Bappon & Roy, 2024. Closest existing work on “consolidating multiple prompts,” but its goal is to make a single prompt to be more efficient by trying to fix the gap of prompt by consolidating them
- **Handoff Debt: The Rediscovery Cost When Coding Agents Take Over Interrupted Tasks** — 2026. Closest framing for agent-to-agent handoff, but in mid-task rather than at the end of project
- **Contextual Memory Virtualisation: DAG-Based State Management and Structurally Lossless Trimming for LLM Agents** — 2026. Alternative solution shape: a live queryable memory store rather than one static portable prompt

These 3 tried the same idea as try to make a single prompt, or try to handoff the project to any other agent using prompt. But it still not our problem

Prompt Loss Definition

- **Survey of Hallucination in Natural Language Generation** — Ji, Lee, Frieske, Yu, Su, Xu, Ishii, Bang, Madotto & Fung, ACM Computing Surveys 2023. Defines extrinsic / input-conflicting hallucination —it was called as the named extrinsic hallucination

- (2) **Extrinsic Hallucinations:** The generated output that cannot be verified from the source content (i.e., output that can neither be supported nor contradicted by the source). For example, in the abstractive summarization task from Table 1, the information “*China has already started clinical trials of the COVID-19 vaccine.*” is not mentioned in source. We can neither find evidence for the generated output from the source nor assert that it is wrong. Notably, the extrinsic hallucination is not always erroneous because it could be from factually correct external information [177, 247]. Such factual hallucination can be helpful because it recalls additional background knowledge to improve the informativeness of the generated text. However, in most of the literature, extrinsic hallucination is still treated with caution because its unverifiable aspect of this additional information increases the risk from a factual safety perspective.

From: survey of hallucination in natural language generation

Evaluate the prompt similarity and code traceability

- **RECOVER: Toward Requirements Generation from Stakeholders' Conversations** — 2024. Extraction methodology this thesis's pipeline adapts for pulling requirements out of dialogue
- **Evaluating the Use of LLMs for Documentation to Code Traceability** — Alor, Khatoonabadi & Shihab, Concordia University, 2025. Precision/Recall/F1 traceability evaluation —

Suggested Solution

- **Spec-Driven Development: From Code to Contract in the Age of AI Coding Assistants – 2026.** Exactly my idea to separate, this paper is about separate the specifications from the code, treat it as the first artifact and then the code will be generated from it

Also in this paper, it was a mention of challenges that we are facing in my thesis

I. INTRODUCTION

For decades, code has been the king of software development. Requirements documents exist, but they drift. Design diagrams are drawn, but they rot. Tests are written, but often after the fact. The code—whatever it actually does—becomes the de facto truth of the system.

This code-centric reality has consequences. When a new developer asks “what should this function do?” the answer is often “read the code.” When a stakeholder asks “does the system meet our requirements?” the answer requires reverse-engineering intent from implementation. When an AI coding assistant is asked to add a feature, it must guess what the developer wants from a vague prompt.

Spec-driven development (SDD) [2], [4] offers an alternative: *make the specification the source of truth, and let code derive from it.* Instead of coding first and documenting later (or never), teams write clear specifications of intended behavior, then generate, implement, or verify code against those specifications. The spec becomes the authoritative description that both humans and machines use to understand, build, and maintain the system.

Consider a developer who prompts an AI with: “Add photo sharing to my app.” The AI must guess: What format? What permissions model? What size limits? Cloud storage or local? Compression? The result is often plausible-looking code that makes dozens of unstated assumptions—many of them wrong. This is what practitioners call “vibe coding”—relying on loose prompts that lead to inconsistent or erroneous outputs from LLMs. By providing AI with unambiguous, executable contracts, SDD enhances the reliability of coding agents and opens new avenues for scalable software creation.

Now consider the same request with a specification: “Users can upload JPEG or PNG photos up to 10MB. Photos are stored in S3 with user-ID-prefixed keys. Only the uploader can delete their photos. Photos are resized to 1024px max dimension on upload.” The AI now has enough information to generate code that matches intent.

Key Insight

The Core Principle: In spec-driven development, code is the implementation detail of the specification—not the other way around. The spec declares intent; the code realizes it.

- **Contextual Memory Virtualisation: DAG-Based State Management and Structurally Lossless Trimming for LLM Agents** — 2026. A memory that store state of the prompt, I can use this idea if storing prompt but keeping track of it mid progress
- **Reverse Prompt Engineering** - Li, Klabjan 2025. A solution to turn output back to prompt using algorithm without input prompt. Like a prediction of prompt when you lost them
 - ⇒ After read these paper, I think that the reserve prompt engineer is more likely not the case for my thesis to use because we already have and still keep the prompts.
 - ⇒ I believe that we should separate the specs from the implemented code to keep the intent of user prompts clean. Also, to keep those specifications clean and not touch by claude, the second paper proposed a structured data to keep instead of raw text.

Research Questions

- **RQ1** — When an AI agent consolidates a prompt sequence without support, does the result contain non traceable requirement, feature loss, or additional information compare with the original prompt?
- **RQ2** — Does the method of splitting between prompts and code help AI agent improve in consolidating the final master prompt?
- **RQ3** — Does a master prompt produced by this pipeline let a different AI model, or a different person, reproduce a functionally equivalent outcome to the original project?

Design of Experiment

Dataset:

1/ I found this dataset it contained around 18,000 prompts from chatgpt responses, we can select some of them that were qualified enough for our thesis

The screenshot shows the Zenodo repository page for the dataset 'DevGPT: Studying Developer-ChatGPT Conversations'. The page includes the title, authors (Xiao, Tao; Treude, Christoph; Hata, Hideaki; Matsumoto, Kenichi), and a description of the dataset. It also features a table of statistics and a list of files.

DevGPT: Studying Developer-ChatGPT Conversations
Xiao, Tao¹; Treude, Christoph²; Hata, Hideaki³; Matsumoto, Kenichi¹

DevGPT is a curated dataset which encompasses 17,913 prompts and ChatGPT's responses including 11,751 code snippets, coupled with the corresponding software development artifacts—ranging from source code, commits, issues, pull requests, to discussions and Hacker News threads—to enable the analysis of the context and implications of these developer interactions with ChatGPT.

Important
Version 9 (2023-11-09) resolves the empty list of conversations attribute: <https://github.com/NAIST-SE/DevGPT/issues/6>

Files

DevGPT.zip

DevGPT

- README.md 10.9 kB
- example.png 608.9 kB
- snapshot_20230727
 - 20230727_195816_fm_sharings.json 16.3 MB
 - 20230727_195927_pr_sharings.json 21.5 MB
 - 20230727_195941_issue_sharings.json 30.4 MB
 - 20230727_195954_discussion_sharings.json 4.4 MB
 - 20230727_200003_commit_sharings.json 15.5 MB
 - 20230727_200102_file_sharings.json 224.0 MB
 - ChatGPT_Link_Sharing.csv 380.6 kB
- snapshot_20230803

Statistics

	All versions	This version
Views	4,931	568
Downloads	1,254	89
Data volume	805.2 GB	102.9 GB

Versions

Version	Date
Version v10	Jul 24, 2025
Version v9	Nov 9, 2023
Version v8	Oct 12, 2023
Version v7	Sep 14, 2023

2/

- My own synthesis examples based on difficulty and the length of prompt:

Run on 3 agents, Antigravity Gemini Flash 3.5, Codex GPT 5.5, Claude Sonnet 4.6

So for this part I think only give example like we did before is just to illustrate the problem, so that we have to conduct experiment between these models (which I owned api key so that I can do it right away)

I can make prompts for 1 simple project (around 10- 15 prompts) and 1 hard project (30+ prompts):

Before the project I will give them this:

Throughout this conversation, treat every request as part of the same software project. Only work within the current project folder and do not create a new project unless I explicitly ask you to.

Acknowledge these instructions and wait for the first prompt.

Project 1:

Prompt 1

Create a portal homepage.

Prompt 2

Add 3 navigation buttons:

Home, Orders, Contact.

Prompt 3

Add dark theme.

Prompt 4

Remove dark theme.

Prompt 5

Add sidebar navigation.

Prompt 6

Undo sidebar.

Prompt 7

Use top navigation instead.

Prompt 8

Change navigation buttons:

Dashboard, Policies, History, Contact.

Prompt 9

Style navigation:

rounded buttons, blue accent, modern font.

Prompt 10

Add a contact form with name, email, and message fields.

The email field must be validated before the form can submit.

Prompt 11

Make sure the page also works well on mobile screens.

For each project (simple and hard), I will do 3 runs:

The prompt will be decided before run the test, before the start, I make sure that the folder was separated from each other models. I will consolidate the prompt as the ground truth for this testing evaluation => **This is the master prompt 1 which is the best matching of the original prompt**

- **1st run:** I feed the prompts to the agent one by one, like a real coding session, until the project is done. And leave it as it is to preserve original prompts with original code
 - **2nd run:** This second run is also like the 1st run but at the end, I will ask Agent to consolidate a master prompt => **got master prompt 2**
 - **3rd run:** my own method. I take the same prompt history and the same final code, but this time I split the work into two parts: first I get an agent to pull out what was actually asked for, looking only at the prompts, without seeing the code. Then I get an agent to look only at the code and describe what it actually does. Then I compare the two lists and combine them into one master prompt, keeping anything that was really asked for and dropping anything the agent just made up on its own. This is my proposed method. => **got master prompt 3**
-
- **4rd run:** Rebuild the project with 3 master prompts for comparison
- ⇒ After I got these master prompts I can do the evaluation by measure and calculate metrics

Metrics to Evaluate the master prompt from models

- **English Prompt — Textual Fidelity**

Automatic metrics:

1. Precision
2. Recall
3. F1
4. BERTScore
5. ROUGE-L

Human evaluation:

1. Traceable Information: Decompose master prompt into individual statements; classify each as traceable if it meets the original requirements
2. Non-Traceable Information: the

I adapted this from the paper about FACTscore which also calculate the percentage of fact that can be drawn from an output generated by llm

- **Functionality of the Code — Functional Fidelity:**

Now if I use the master prompt to rebuilt the product I have to check for

Each feature in the original's checklist is checked against the rebuilt product: Match / Missing / Extra / Modified. Higher is better.

Based on the paper: [Evaluating the Use of LLMs for Documentation to Code Traceability](#)

There is also another way to conduct this function test is to make unit test for each requirements and measure the percentage of

passed unit test / total tests